Week 3 - Extending CloudFormation (custom resources, macros, modules, Hooks, Registry)

DOP-C02 Domain 2 (Configuration Management and IaC). Plain CloudFormation creates only **supported resource types** from the **template as written**. These five features extend it.

Feature	What it does	When it runs	Exam keyword
Custom resource	Runs **your code** (Lambda or SNS) as a resource during Create / Update / Delete	During stack operations	Something CloudFormation can't do, look up a value, empty a bucket on delete
Macro	**Rewrites the template** (or a snippet) before processing	**Before** the stack operation	Template shorthand, boilerplate, `Transform:`
Module	A **reusable bundle of resources** used as one type (`...::MODULE`)	Expanded at processing time, **no separate stack**	Reusable approved pattern without nested stacks
Hook	**Checks configuration before** create / update / delete, then **fails or warns**	**Before provisioning**	Proactive control, block non-compliant resources
Registry	Catalog of **extensions**: resource types, modules, hooks; **public** or **private**	Management plane	Third-party resource, activate extension, private resource type

Part 1 - Custom resources (Lambda-backed)

How they work

```
Resources:
  EmptyBucketOnDelete:
    Type: Custom::EmptyBucket                  # or AWS::CloudFormation::CustomResource
    Properties:
      ServiceToken: !GetAtt EmptyBucketFunction.Arn   # Lambda ARN (or SNS topic ARN). Required
      ServiceTimeout: 300                      # optional, seconds to wait for a response (max 3600)
      BucketName: !Ref DataBucket              # your own properties, passed to the function
      Version: "2"                             # change this to force an Update call

Outputs:
  ResultValue:
    Value: !GetAtt EmptyBucketOnDelete.SomeKey  # values returned in the response "Data"
```

1. On stack **Create, Update or Delete**, CloudFormation sends a **request event** to the `ServiceToken` (**invokes Lambda** or **publishes to SNS**).
2. Your code works, then **PUTs a response** to the **presigned S3** `ResponseURL` in the event.
3. CloudFormation waits for the response, then marks the resource **complete or failed**.

Request event fields

Field	Meaning
`RequestType`	`Create`, `Update` or `Delete`
`ResponseURL`	Presigned S3 URL for the result
`StackId`, `RequestId`, `LogicalResourceId`, `ResourceType`	Identify the request; echo them back
`PhysicalResourceId`	On **Update** and **Delete**: the ID you returned earlier
`ResourceProperties`	Your properties (values arrive as **strings**)
`OldResourceProperties`	On **Update**: previous properties

Response body (PUT to ResponseURL)

```
{
  "Status": "SUCCESS",
  "Reason": "See CloudWatch log stream ...",
  "PhysicalResourceId": "my-unique-id",
  "StackId": "...", "RequestId": "...", "LogicalResourceId": "...",
  "NoEcho": false,
  "Data": { "SomeKey": "value" }
}
```

- `Status` is `SUCCESS` or `FAILED` (with a `Reason`). Keep `PhysicalResourceId` **stable** unless you mean replacement. `NoEcho: true` masks Data.
- Response body max **4,096 bytes**. **Data** values are read with `Fn::GetAtt CustomResource.<Key>`.

Inline Python example (cfnresponse)

```
EmptyBucketFunction:
  Type: AWS::Lambda::Function
  Properties:
    Runtime: python3.12
    Handler: index.handler
    Timeout: 300
    Role: !GetAtt EmptyBucketRole.Arn
    Code:
      ZipFile: |
        import boto3, cfnresponse
        def handler(event, context):
            try:
                if event["RequestType"] == "Delete":
                    b = boto3.resource("s3").Bucket(event["ResourceProperties"]["BucketName"])
                    b.object_versions.delete()          # empty the bucket (all versions)
                cfnresponse.send(event, context, cfnresponse.SUCCESS, {},
                                 event.get("PhysicalResourceId", "empty-bucket-helper"))
            except Exception as e:
                cfnresponse.send(event, context, cfnresponse.FAILED, {"Error": str(e)},
                                 event.get("PhysicalResourceId", "empty-bucket-helper"),
                                 reason=str(e))
```

- `cfnresponse` is available automatically **only for inline** `ZipFile` **code** (Python, Node.js). For packaged code use your own helper, **crhelper** (Python), or the **CDK Provider framework**.

Rules behind most exam questions

1. **Always respond**, for **every** RequestType, even on errors (try / except / finally). No response means the stack **hangs** in CREATE / UPDATE / DELETE_IN_PROGRESS until the timeout: **1 hour by default**, or `ServiceTimeout`. Rollback can hang too, because it sends a Delete to the same broken function.
2. **Delete must succeed.** A failing or silent Delete gives `DELETE_FAILED` and stuck rollbacks. Treat "resource already gone" as **success**.
3. `PhysicalResourceId`**:** a **different** ID on Update means **replacement**, and CloudFormation later sends a **Delete for the old ID**. A new random ID on every update causes **unexpected deletes**.
4. **Update only runs when properties change.** Change a `Version` or timestamp property to force a rerun; changing only the Lambda code doesn't.
5. **Network:** the function must reach the **S3 presigned URL**. A **VPC Lambda** needs **NAT or an S3 gateway endpoint**.
6. Give the Lambda enough timeout and log requests to **CloudWatch Logs**.
7. **SNS-backed** custom resources let **external systems** (on-premises, another account) handle requests.

Typical uses

- **Empty an S3 bucket** before deletion (avoid `DELETE_FAILED`).
- **Look up values:** AMIs with custom filters, values from **another Region or account**, third-party APIs.
- **Unsupported APIs** or **third-party SaaS**.
- **Seed a database**, external DNS, random strings or passwords (stored in Secrets Manager).
- **Deploy-time steps** (start a CodeBuild job, invalidate CloudFront).

- **Custom resource vs registry resource type:** custom resources are quick; a registry **resource type** is first-class with **drift detection**, **import**, a schema and reuse. Choose it for long-lived, widely used integrations.

Part 2 - Macros (template transforms)

A macro **receives a template or fragment and returns a modified version** before CloudFormation creates the change set or stack.

AWS-managed transforms

Transform	What it does
`AWS::Serverless-2016-10-31` (SAM)	Expands `AWS::Serverless::Function`, `Api`, `SimpleTable`, `Application` into full resources
`AWS::Include`	Inserts a **template snippet from S3** (`Fn::Transform` with `Name: AWS::Include`, `Parameters: Location: s3://...`)
`AWS::LanguageExtensions`	`Fn::ForEach` (loops), `Fn::Length`, `Fn::ToJsonString`, **FindInMap** `DefaultValue`, intrinsic functions in DeletionPolicy / UpdateReplacePolicy
`AWS::SecretsManager-2020-07-23`	Creates a **rotation Lambda** for `AWS::SecretsManager::RotationSchedule` (`HostedRotationLambda`)
`AWS::CodeDeployBlueGreen`	**ECS blue/green** through CloudFormation (`AWS::CodeDeploy::BlueGreen` hook)

Custom macros

```
# 1. Register the macro (in the SAME account and Region as the stacks that use it)
Resources:
  TagMacroFunction:
    Type: AWS::Lambda::Function
    Properties: { ... }          # returns the modified fragment
  TagMacro:
    Type: AWS::CloudFormation::Macro
    Properties:
      Name: AddStandardTags
      FunctionName: !GetAtt TagMacroFunction.Arn
```

```
# Lambda for the macro
def handler(event, context):
    fragment = event["fragment"]                # the template or snippet
    for r in fragment.get("Resources", {}).values():
        r.setdefault("Properties", {}).setdefault("Tags", []).append(
            {"Key": "CostCenter", "Value": event["templateParameterValues"].get("CostCenter", "unknown")})
    return {"requestId": event["requestId"], "status": "success", "fragment": fragment}
```

```
# 2. Use it
Transform: [AWS::LanguageExtensions, AddStandardTags]   # template-level, processed in the listed order
Resources:
  Bucket:
    Type: AWS::S3::Bucket
    Properties:
      Fn::Transform:                                    # snippet-level
        Name: SetBucketDefaults
        Parameters: { Encryption: aws:kms }
```

Rules

- Deploying needs `CAPABILITY_AUTO_EXPAND` (also for nested stacks containing macros).
- Macros must be **registered in the same account and Region** as the stack. Use **StackSets** to deploy them widely.
- The Lambda receives `fragment`, `params`, `templateParameterValues`, `region`, `accountId`, `requestId`, and returns `requestId`, `status: success`, **and the new** `fragment`.
- **Template-level** transforms (`Transform:`) run **in the listed order**; **snippet-level** use `Fn::Transform`.

- **Change sets** show the **processed** template; the console shows **original vs processed**.
- Macros run **every time** the template is processed: keep them **deterministic**.
- Typical uses: org-wide **tags and defaults**, **generating repeated resources** (`Fn::ForEach` now covers many loops), string functions, expanding a **shorthand DSL**.

Part 3 - Modules

A **module** is a reusable, versioned **template fragment** (resources, parameters, outputs) registered in the **Registry** as type `MODULE` and used like a resource:

```
Resources:
  LogsBucket:
    Type: MyCompany::S3::SecureBucket::MODULE      # Organization::Service::Name::MODULE
    Properties:
      BucketPurpose: logs                          # module parameters
      RetentionDays: 365
  Reader:
    Type: AWS::IAM::Policy
    Properties:
      PolicyDocument:
        Statement:
          - Effect: Allow
            Action: s3:GetObject
            Resource: !Sub "${LogsBucketBucket.Arn}/*"   # module resource =
    <ModuleLogicalId><ResourceLogicalId>
```

- Module resources are **inserted into your stack**: **no separate stack** (unlike nested stacks).
- **Logical IDs are prefixed** with the module's logical ID: `Bucket` inside `LogsBucket` becomes `LogsBucketBucket`.
- Build with the **CloudFormation CLI** (`cfn init`, choose module, write `fragments/`, `cfn submit`): a **private extension** in that **account and Region**.
- **Versioned** with a **default version**; stacks pick up new versions on their **next update**. Modules can contain modules.
- Bake in security defaults (encryption, public access block, logging) once for every team.
- Register across accounts and Regions with **StackSets** (`AWS::CloudFormation::ModuleVersion` / `ModuleDefaultVersion`).

Modules vs nested stacks

	Module	Nested stack
Result	Resources **merged into the same stack**	A **separate child stack**
Resource limit	Counts toward the **same stack's 500**	Each child has **its own 500**
Distribution	**Registry** (versioned, per account / Region)	Template in **S3** (`TemplateURL`)
Referencing internals	Direct `Ref` / `GetAtt` to `ModuleIdResourceId`	Only through **child Outputs**
Best for	Small standard building blocks	Big components, large apps, past 500 resources

Part 4 - Hooks (proactive controls)

A **Hook** inspects resources **before CloudFormation creates, updates or deletes them**, then **stops the operation** (`FailureMode: FAIL`) or **warns** (`WARN`).

Type	How you write it
Guard Hook	**CloudFormation Guard** rules stored in **S3**. **No code**
Lambda Hook	CloudFormation **invokes your Lambda**, which returns compliant or not
Custom Hook (registry extension)	Built with the **CloudFormation CLI** (Python / Java), handlers `preCreate`, `preUpdate`, `preDelete`; private or public third-party

- **Targets: resources** (specific types, before create / update / delete), **stacks** (whole template), **change sets**, and **Cloud Control API** operations.
- **Configuration per account and Region:** `TargetStacks: ALL` (or NONE), filters by stack name, stack role, resource type; `FailureMode: FAIL` or `WARN` (use WARN to roll out new rules safely).
- Activate in **each account and Region**; roll out org-wide with **StackSets**.
- **AWS Control Tower proactive controls** are built on Hooks.
- Results show in **stack events** ("Hook ... failed: ...").

```
# Guard rule (stored in S3) for a Guard Hook: buckets must block public access and use KMS
let buckets = Resources.*[ Type == 'AWS::S3::Bucket' ]
rule s3_secure when %buckets !empty {
  %buckets.Properties.PublicAccessBlockConfiguration.BlockPublicAcls == true
  %buckets.Properties.BucketEncryption.ServerSideEncryptionConfiguration[*].ServerSideEncryptionByDefault.SS
    EAlgorithm == 'aws:kms'
}
```

Hooks vs other controls

Control	When	Scope	Blocks?
cfn-lint	Pipeline / IDE, before deploy	Template syntax and spec	Fails the build
cfn-guard in CI	**Pipeline**, before deploy (shift left)	Your rules	Fails the build, only if that pipeline runs it
CloudFormation Hooks	**At deploy time inside CloudFormation**, before provisioning	**Every stack** in the account / Region, whoever deploys	**Yes** (FAIL) or warn
SCPs / IAM	Every API call	All tools	Yes (deny)
AWS Config rules	**After** resources exist (detective)	All resources	No, detects (plus remediation)
Control Tower controls	Preventive (SCP), **proactive (Hooks)**, detective (Config)	Organization	Depends on type

- Block non-compliant resources for every stack before creation: **Hooks**. Detect existing ones: **Config**. Block in one pipeline: **cfn-guard / cfn-lint**.

Part 5 - The CloudFormation Registry

Extension type	Example	Notes
Resource types	`AWS::S3::Bucket` (AWS), `MongoDB::Atlas::Cluster`, `Datadog::Monitors::Monitor` (third-party), `MyCo::Network::Firewall` (private)	Handlers create, read, update, delete, list; support **drift detection, import, change sets**
Modules	`MyCo::S3::SecureBucket::MODULE`	Part 3
Hooks	Your own or third-party compliance hooks	Part 4

	Public	Private
Publisher	**AWS** or **verified third parties**	**You**
How to use	**Activate** in each account and Region (`ActivateType`); optional **auto-update of minor versions**	**Register** from your code (`cfn submit` / `RegisterType`)
Visibility	All customers	Your account and Region (unless published)
Roles	**Execution role** for handlers + **logging config** (CloudWatch Logs)	Same

Building a private resource type

1. `cfn init` (resource / module / hook; Python, Java, Go, TypeScript).
2. Edit the **schema** (properties, primary identifier, read-only properties, handler permissions).
3. Write handlers: **create, read, update, delete, list**.
4. `cfn test` (contract tests), `cfn submit`, set the **default version**.
5. Use it like any type: `Type: MyCo::Network::Firewall`.

- **Many accounts and Regions:** StackSets with `AWS::CloudFormation::TypeActivation` (public), `ResourceVersion` / `ResourceDefaultVersion` (private), `HookVersion` / `HookDefaultVersion`, `ModuleVersion`.
- **Third-party SaaS** (MongoDB Atlas, Datadog): **activate the public extension**, give its execution role and API keys (Secrets Manager); better than writing your own custom resource.

Part 6 - Decision guide

Requirement	Choose
Delete all objects from a bucket when the stack is deleted	**Custom resource** (Lambda on Delete)
Read a value from another Region or account during deployment	**Custom resource**
One-off API call or step CloudFormation doesn't support	**Custom resource**
Reusable first-class integration with drift and import	**Private registry resource type**
Third-party SaaS resource in templates	**Activate a public third-party extension**
Shorthand or standard tags added at template processing	**Macro**
Loop to create N similar resources	`AWS::LanguageExtensions` `Fn::ForEach` (or a macro)
Insert a shared snippet from S3	`AWS::Include`
Reuse an approved secure-bucket pattern **without** extra stacks	**Module**
Reuse a big component, or split a 700-resource app	**Nested stacks**
Block non-compliant resources **before** creation for **every** deployment	**CloudFormation Hook**
Detect non-compliance **after** deployment	**AWS Config**
Check templates in the pipeline	**cfn-lint / cfn-guard**

Part 7 - Exam traps

Trap	Truth
The custom resource Lambda failed, so the stack fails right away	Without a **response** the stack **waits up to 1 hour** (or `ServiceTimeout`). **Always respond**
Stack stuck in DELETE_IN_PROGRESS on a custom resource	**Delete handler** didn't respond or failed; treat not found as success
Custom resource Lambda in a VPC hangs	No route to the **S3 presigned URL**: **NAT or S3 endpoint**
Changing the Lambda code reruns the custom resource	**No.** Change a **property** (`Version`)
New PhysicalResourceId on every update	Treated as **replacement**; old ID gets a **Delete**
`cfnresponse` is in any Lambda package	Only **inline** `ZipFile`; otherwise crhelper or own helper
Read custom resource results	`!GetAtt CustomRes.Key` from `Data` (`NoEcho` to mask)
Stack with a macro fails on capabilities	`CAPABILITY_AUTO_EXPAND`
Use a macro registered in another account	**Same account and Region** only; deploy with **StackSets**
Need loops in templates	`Fn::ForEach` (LanguageExtensions)
Insert a snippet from S3	`AWS::Include`
Modules create nested stacks	**No**, merged into the same stack with prefixed IDs
Reference a resource inside a module	`<ModuleLogicalId><ResourceLogicalId>` (`LogsBucketBucket`)
Modules help past 500 resources	**No**; use **nested stacks**
Block unencrypted buckets for all stacks regardless of pipeline	**Hooks**, not only cfn-guard in CI
Config rules prevent creation	Config is **detective**; Hooks **proactive**; SCPs **preventive**
Test a new Hook without breaking deployments	`FailureMode: WARN` first
Use a public third-party type immediately	**Activate** it per account and Region with an execution role
Widely reused integration with drift support	**Registry resource type**
Control Tower proactive controls	Built on **Hooks**

Part 8 - Hands-on lab (about 90 minutes, very low cost)

Lab 1 - Custom resource that empties a bucket and returns data

1. Stack with an S3 bucket, the `EmptyBucketFunction` from Part 1 (inline, `cfnresponse`) with a role allowing `s3:ListBucket*`, `s3:DeleteObject*` and logs, and a `Custom::EmptyBucket` with `ServiceToken`, `BucketName: !Ref DataBucket`, `ServiceTimeout: 300`.
2. Return `Data: {"Greeting": "hello"}` on Create and output `!GetAtt EmptyBucketOnDelete.Greeting`.
3. Upload objects, **delete the stack**: the bucket is emptied and the stack deletes cleanly. Read the CloudWatch Logs.
4. Break it: remove `cfnresponse.send` from the Delete path, recreate, add objects, delete: the stack **waits until** `ServiceTimeout` (5 minutes, not 1 hour). Fix the code.

Lab 2 - Force an update

1. Update with **only the Lambda code** changed: the custom resource **doesn't run**.
2. Change `Version: "3"`: an **Update** event arrives with `OldResourceProperties`.

Lab 3 - LanguageExtensions and Include

1. `Transform: AWS::LanguageExtensions` and `Fn::ForEach` to create 3 SQS queues from a list parameter `[orders, billing, email]`.
2. Upload a snippet (one SNS topic) to S3 and insert it with `AWS::Include`.
3. Deploy **without** `CAPABILITY_AUTO_EXPAND` (fails), then with it. Compare **original** and **processed** templates in the console.

Lab 4 - Custom macro

1. Deploy the Lambda + `AWS::CloudFormation::Macro` `AddStandardTags` (Part 2).
2. Deploy a stack with `Transform: [AddStandardTags]` and two buckets: each gets a `CostCenter` tag. Check the processed template.

Lab 5 - Guard Hook in WARN, then FAIL

1. Upload the Guard rule (Part 4) to S3.
2. Console **Hooks, Create a Hook with Guard**, target `AWS::S3::Bucket` create and update, **FailureMode WARN**, TargetStacks ALL.
3. Deploy an unencrypted public bucket: **succeeds** with a **warning** in events.
4. Switch to **FAIL** and redeploy: fails **before the bucket is created**.

Lab 6 (optional) - Registry

1. Browse **Registry, Public extensions** (third-party), open a Datadog or MongoDB type, note **Activate** and the execution role.
2. Optional: `pip install cloudformation-cli cloudformation-cli-python-plugin`, `cfn init` a **module** with a secure bucket fragment, `cfn submit`, use `MyCo::S3::SecureBucket::MODULE`, reference `<ModuleId>Bucket`.

Clean up

- Delete the lab stacks, deactivate the Hook, delete the macro stack, deregister module or type versions, remove the S3 snippet and rule files.

Part 9 - Check yourself

Questions

1. What are the two possible targets for a custom resource's `ServiceToken`?
2. Name the three `RequestType` values.
3. A custom resource Lambda throws and never responds. What happens, and how do you prevent it?
4. How does CloudFormation receive the result, and how do other resources read returned values?
5. You changed the Lambda code but the custom resource didn't run on update. Why, and how do you force it?
6. What happens if a custom resource returns a different `PhysicalResourceId` on Update?
7. A Lambda-backed custom resource in a VPC always times out. Likely cause?
8. Which transform adds `Fn::ForEach`? Which inserts a snippet from S3?
9. Which capability is required for macros or `AWS::Serverless`?
10. Where must a custom macro be registered relative to the stacks using it?
11. How are module resources named in the final stack? Do modules create separate stacks?
12. When choose a module instead of a nested stack, and the reverse?
13. The company must stop **any** CloudFormation deployment from creating security groups open to 0.0.0.0/0 on port 22, before creation, whatever pipeline is used. What do you use and how do you roll it out safely?
14. Difference between Hooks, cfn-guard in CI, Config rules and SCPs?
15. A team wants to manage MongoDB Atlas clusters from CloudFormation. Simplest approach?
16. When is a private registry resource type better than a custom resource?

Answers

1. A **Lambda function ARN** or an **SNS topic ARN**.
2. **Create, Update, Delete**.
3. The stack **waits** until the timeout (**1 hour default**, or `ServiceTimeout`), then fails; rollback may hang on Delete. **Always send SUCCESS or FAILED** in try / except / finally (cfnresponse, crhelper), with a sensible `ServiceTimeout`.
4. A **JSON PUT to the presigned S3** `ResponseURL`; `Data` values via `!GetAtt CustomResource.Key`.
5. Updates happen **only when the custom resource's properties change**; change a `Version` or timestamp property.
6. Treated as **replacement**: CloudFormation sends a **Delete for the old physical ID** during cleanup.
7. **No route to the S3 presigned URL** (no NAT or S3 endpoint).
8. `AWS::LanguageExtensions`; `AWS::Include`.
9. `CAPABILITY_AUTO_EXPAND`.
10. **Same account and Region**; StackSets for many.
11. **Merged into the same stack**, IDs **prefixed with the module logical ID** (`LogsBucketBucket`). **No separate stacks.**
12. **Module:** small standard building blocks, Registry-versioned, direct references. **Nested stack:** large components, split big apps, pass **500 resources**.
13. A **CloudFormation Hook** (Guard or Lambda) on `AWS::EC2::SecurityGroup`, activated everywhere with **StackSets**; start with `WARN`, then **FAIL**.
14. **Hooks:** proactive, inside CloudFormation for every stack. **cfn-guard in CI:** shift-left in one pipeline. **Config:** detective after creation. **SCPs:** preventive at the API level for all tools.
15. **Activate the public third-party** `MongoDB::Atlas::*` **resource type** with its execution role and API keys (Secrets Manager), then use it in templates.
16. For **long-lived, widely reused** integrations needing **drift detection, import, a schema, versioning and full handlers**.

Quick memory summary

- **Custom resource:** `Custom::X` + **ServiceToken** (Lambda / SNS); Create / Update / Delete events; **PUT to presigned S3 ResponseURL**; **always respond** (1 h default wait, `ServiceTimeout`); Delete must succeed; stable PhysicalResourceId; Update only on property change; `GetAtt .Data`; VPC Lambda needs NAT / S3 endpoint; `cfnresponse` inline only.
- **Macros:** AWS transforms (**Serverless, Include, LanguageExtensions, SecretsManager, CodeDeployBlueGreen**); custom `AWS::CloudFormation::Macro` (same account + Region); **CAPABILITY_AUTO_EXPAND**; processed template in change sets.
- **Modules:** `Org::Service::Name::MODULE`, merged into the same stack, **prefixed IDs**, Registry-versioned, not a fix for 500 resources.
- **Hooks:** Guard / Lambda / custom; pre-create / update / delete; resources, stacks, change sets; **FAIL or WARN**; proactive; Control Tower proactive controls.
- **Registry:** resource types, modules, hooks; **public (activate)** vs **private (register)**; execution role; StackSets to roll out.